

Back Up a Shared Server in MongoDB

Continuing the series on MongoDB, in this article, readers learn how to set up a backup for the sharded environment that was set up over the previous two articles.

In the previous article in this series, we set up a sharded environment in MongoDB. This article deals with one of the most intriguing and crucial topics in database administration—backups. The article will demonstrate the MongoDB backup process and will make a backup of the sharded server that was configured earlier. So, to proceed, you must set up your sharded environment as per our previous article as we'll be using the same configuration.

Before we move on with the backup, make sure that the balancer is not running.

The balancer is the process that ensures that data is distributed evenly in a sharded cluster. This is an automated process in MongoDB and at most times, you won't be bothered with it. In this case, though, it needs to be stopped so that no chunk migration takes place while we back up the server. If you're wondering what the term 'chunk migration' means, let me tell you that if one

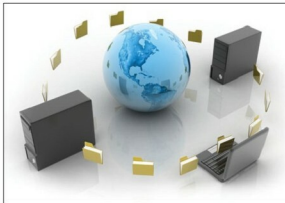
shard in a sharded MongoDB environment has more data stored than its peers, then the balancer process migrates some data to other shards. Evenly distributed data ensures optimal performance in a sharded environment.

So now connect to a Mongo process by opening a command prompt, going to the MongoDB root directory and typing 'Mongo'. Type `sh.getBalancerState()` to find out the balancer's status. If you get `true` as the output, your balancer is running. Type `sh.stopBalancer()` to stop the balancer.

The next step is to back up the config server, which stores metadata about shards. In the previous article, we set up three config servers for our shard. Since all the

config servers store the same metadata and since we have three of them just to ensure availability, we'll be backing just one config server for demonstration purposes. So open a command prompt and type the following command to back up the config database of our config server:

```
C:\Users\viny\Desktop>mongodb-win32-x86_2.6.0\bin>mongodump --host localhost:59020 --db config
```



This command will dump your config database under the dump directory of your MongoDB root directory.

Now let's back up our actual data by taking backups of all of our shards. Issue the following commands, one by one, and take a backup of all the three replica sets of both the shards that we configured earlier:

```
mongodump --host localhost:38020 --out .\shard1\replica1
mongodump --host localhost:38021 --out .\shard1\replica2
mongodump --host localhost:38022 --out .\shard1\replica3
mongodump --host localhost:48020 --out .\shard2\replica1
mongodump --host localhost:48021 --out .\shard2\replica2
mongodump --host localhost:48022 --out .\shard2\replica3
```

The `--out` parameter defines the directory where MongoDB will place the dumps. Now you can start the balancer by issuing the `sh.startBalancer()` command and resume normal operations. So we're done with our backup operation.

If you want to explore a bit more about backups and restores in MongoDB, you can check MongoDB documentation and the article in <http://www.thegeekstuff.com/2013/09/mongodump-mongorestore/> which will give you some good insights into Mongodump and Mongorestore commands. 

By: Vinayak Pandey

The author is an experienced database developer, with exposure to various database and data warehousing tools and techniques, including Oracle, Teradata, Informatica PowerCenter and MongoDB.

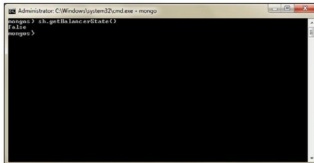


Figure 1: Balancer status